GIT · AIDE-MÉMOIRE · PRÉPAVENIR, OCTOBRE 2026

# Aide-mémoire Git

Toutes les commandes utiles de la semaine, rangées par usage. Les mots entre `< >` sont à remplacer : `<fichier>` par un nom de fichier, `<id>` par un identifiant de commit (les 7 premiers caractères suffisent), `<branche>` par un nom de branche. `HEAD` désigne le dernier commit de la branche où vous êtes, `HEAD~1` celui d'avant, `HEAD~2` celui encore avant.

> **Les trois zones.** Vos fichiers (répertoire de travail) → `git add` → l'index (ce qui partira dans le prochain commit) → `git commit` → le dépôt (l'historique). Puis `git push` envoie l'historique sur GitHub, et `git pull` le récupère.

## Configurer (une seule fois par machine)

| Commande | À quoi ça sert |
|---|---|
| `git config --global user.name "Prénom Nom"` | le nom qui signera vos commits |
| `git config --global user.email "vous@mail.fr"` | l'adresse des commits, la même que sur GitHub |
| `git config --global init.defaultBranch main` | appeler la branche principale `main` |
| `git config --global core.editor nano` | utiliser nano au lieu de vim pour les messages |
| `git config --global pull.rebase false` | `git pull` fusionne au lieu de s'arrêter sur une erreur |
| `git config --global --list` | voir toute sa configuration |

## Démarrer un projet

| Commande | À quoi ça sert |
|---|---|
| `git init` | transformer le dossier courant en dépôt Git (crée le dossier caché `.git`) |
| `git clone <adresse>` | télécharger un dépôt existant, avec tout son historique |
| `git clone <adresse> <dossier>` | pareil, en choisissant le nom du dossier créé |

## Enregistrer son travail

| Commande | À quoi ça sert |
|---|---|
| `git status` | voir où on en est : fichiers modifiés, ajoutés, non suivis. **À taper tout le temps.** |
| `git status -s` | la même chose en version courte ( `??` non suivi, `M` modifié, `A` ajouté) |
| `git add <fichier>` | mettre un fichier dans l'index, prêt pour le prochain commit |
| `git add .` | tout ajouter dans le dossier courant (vérifiez avec `git status` avant de commiter) |
| `git add -p` | choisir morceau par morceau ce qu'on ajoute, en répondant `y` ou `n` |
| `git commit -m "Message"` | enregistrer un commit avec ce qui est dans l'index |
| `git commit -am "Message"` | ajouter **et** commiter d'un coup les fichiers déjà suivis (pas les nouveaux) |
| `git mv <ancien> <nouveau>` | déplacer ou renommer un fichier, en gardant son historique |
| `git rm <fichier>` | supprimer un fichier et enregistrer sa suppression |
| `git rm --cached <fichier>` | arrêter de suivre un fichier sans le supprimer du disque (avant de l'ajouter au `.gitignore` ) |

## Comparer

| Commande | À quoi ça sert |
|---|---|
| `git diff` | ce qui a changé et n'est pas encore dans l'index |
| `git diff --staged` | ce qui est dans l'index, donc ce qui partira dans le commit |
| `git diff <id1> <id2>` | les différences entre deux commits |
| `git diff main <branche>` | les différences entre deux branches |

## Lire l'historique

| Commande | À quoi ça sert |
|---|---|
| `git log` | l'historique complet : identifiant, auteur, date, message |
| `git log --oneline` | une ligne par commit |
| `git log --oneline --graph --all` | toutes les branches dessinées, avec leurs fusions |
| `git log --stat` | avec la liste des fichiers modifiés par chaque commit |
| `git log -p -- <fichier>` | chaque modification d'un fichier, ligne par ligne |
| `git log --oneline -- <fichier>` | les commits qui ont touché un fichier, même supprimé depuis |
| `git log --follow --oneline -- <fichier>` | pareil, en suivant le fichier à travers ses renommages |
| `git log --author="Alice"` | les commits d'une personne |
| `git log --since="2 weeks ago"` | les commits récents |
| `git log -S "texte" --oneline` | les commits qui ont ajouté ou retiré ce texte |
| `git log --diff-filter=D --name-only --oneline` | les commits qui ont supprimé des fichiers, et lesquels |
| `git log --format="%h %an %ad %s" --date=short` | une ligne par commit avec l'auteur et la date |
| `git show <id>` | le détail d'un commit : message et modifications |
| `git show <id>:<fichier>` | le contenu d'un fichier tel qu'il était dans ce commit |
| `git blame <fichier>` | pour chaque ligne : qui l'a écrite en dernier, et dans quel commit |
| `git shortlog -sn HEAD` | le nombre de commits par auteur |

## Annuler et réparer

| Commande | À quoi ça sert |
|---|---|
| `git restore <fichier>` | annuler les modifications d'un fichier depuis le dernier commit. **Elles sont perdues.** |
| `git restore --staged <fichier>` | sortir un fichier de l'index, sans perdre les modifications |
| `git restore --source=<id> <fichier>` | remettre un fichier dans l'état d'un ancien commit |
| `git commit --amend -m "Nouveau message"` | corriger le message du dernier commit |
| `git commit --amend --no-edit` | ajouter au dernier commit un fichier oublié (après un `git add`) |
| `git reset --soft HEAD~1` | défaire le dernier commit, en gardant ses modifications dans l'index |
| `git revert <id>` | annuler un commit en créant un nouveau commit inverse. **La bonne méthode une fois le commit poussé.** |

**Attention.** `--amend` et `reset` réécrivent l'historique : ne les utilisez que sur des commits **pas encore poussés**. Pour un commit déjà sur GitHub, utilisez `git revert`. Et méfiez-vous de `git reset --hard` et `git push --force`, que vous croiserez sur Internet : ils effacent du travail sans retour possible.

## Les branches

| Commande | À quoi ça sert |
|---|---|
| `git branch` | lister les branches (l'étoile montre celle où vous êtes) |
| `git branch -a` | lister aussi les branches de GitHub ( `remotes/origin/...` ) |
| `git branch -vv` | les branches avec leur dernier commit et leur lien avec GitHub |
| `git switch -c <branche>` | créer une branche et aller dessus |
| `git switch <branche>` | changer de branche |
| `git switch -` | revenir à la branche précédente |
| `git branch -m <ancien> <nouveau>` | renommer une branche |
| `git branch --merged` / `--no-merged` | les branches déjà fusionnées, ou pas encore |
| `git branch -d <branche>` | supprimer une branche fusionnée |
| `git branch -D <branche>` | supprimer une branche même non fusionnée (attention) |
| `git switch --detach <id>` | aller voir un ancien commit (*detached HEAD*). Revenir avec `git switch main`. |

## Fusionner et résoudre les conflits

| Commande | À quoi ça sert |
|---|---|
| `git merge <branche>` | fusionner une branche dans la branche où vous êtes |
| `git merge --no-edit <branche>` | pareil, en gardant le message de fusion proposé |
| `git merge --abort` | abandonner une fusion en conflit, tout revient comme avant |
| `git cherry-pick <id>` | copier un seul commit d'une autre branche sur la branche courante |
| `git cherry-pick --abort` | abandonner un cherry-pick en conflit |

Dans un conflit, le fichier contient `<<<<<<< HEAD` (votre version), `=======`, puis `>>>>>>> <branche>` (l'autre version). Gardez ce que vous voulez, **supprimez les trois marqueurs**, puis `git add <fichier>` et `git commit`.

## GitHub et les dépôts distants

| Commande | À quoi ça sert |
|---|---|
| `git remote add origin <adresse>` | relier le dépôt local à un dépôt GitHub |
| `git remote -v` | voir à quelle adresse le dépôt est relié |
| `git remote set-url origin <adresse>` | changer cette adresse (par exemple de HTTPS vers SSH) |
| `git push -u origin <branche>` | premier envoi d'une branche sur GitHub |
| `git push` | envoyer ses commits (une fois la branche reliée) |
| `git pull` | récupérer les commits des autres et les fusionner |
| `git fetch` | récupérer les nouveautés de GitHub sans rien fusionner |
| `git pull origin main` | fusionner le `main` de GitHub dans la branche courante (pour régler un conflit de PR) |
| `git push origin --delete <branche>` | supprimer une branche sur GitHub |
| `ssh -T git@github.com` | vérifier que la clé SSH fonctionne (« Hi pseudo! ») |

## Mettre de côté, retrouver, marquer

| Commande | À quoi ça sert |
|---|---|
| `git stash` | mettre de côté les modifications en cours, le dossier redevient propre |
| `git stash push -m "description"` | pareil, avec une description |
| `git stash list` | voir ce qui a été mis de côté |
| `git stash pop` | ressortir la dernière mise de côté, et la retirer de la liste |
| `git stash apply` | la ressortir en la gardant dans la liste |
| `git stash drop` | supprimer la dernière mise de côté |
| `git reflog` | le journal de bord local : retrouver un commit « perdu » (branche supprimée, reset) |
| `git branch <nom> <id>` | recréer une branche sur un commit retrouvé |
| `git tag -a v1.0 -m "Description"` | poser une étiquette de version sur le commit courant |
| `git tag` | lister les étiquettes |
| `git push origin v1.0` / `git push --tags` | envoyer une étiquette, ou toutes, sur GitHub (un `git push` simple ne les envoie pas) |

## Ranger le dossier

| Commande | À quoi ça sert |
|---|---|
| `.gitignore` | un fichier texte qui liste ce que Git doit ignorer : `notes.txt`, `*.log`, `tmp/` |
| `git ls-files` | lister les fichiers suivis par Git |
| `git clean -n` | lister les fichiers non suivis qui seraient supprimés |
| `git clean -f` | supprimer vraiment ces fichiers non suivis (vérifiez avec `-n` avant) |
| `.gitkeep` | un fichier vide, par convention, pour qu'un dossier vide soit suivi par Git |

## J'ai fait une bêtise : que faire ?

| Situation | Solution |
|---|---|
| J'ai cassé un fichier, je n'ai rien commité | `git restore <fichier>` |
| J'ai fait `git add` d'un fichier par erreur | `git restore --staged <fichier>` |
| J'ai fait une faute dans le message du dernier commit (pas encore poussé) | `git commit --amend -m "Bon message"` |
| J'ai oublié un fichier dans le dernier commit (pas encore poussé) | `git add <fichier>`, puis `git commit --amend --no-edit` |
| J'ai commité trop vite, je veux retravailler (pas encore poussé) | `git reset --soft HEAD~1` |
| J'ai poussé un commit qui casse tout | `git revert <id>`, puis `git push` |
| J'ai commité un mot de passe | le retirer ne suffit pas, il reste dans l'historique : **changez le mot de passe**, puis ajoutez le fichier au `.gitignore` |
| `git push` est refusé ( `fetch first` ) | `git pull`, régler les conflits éventuels, puis `git push` |
| `refusing to merge unrelated histories` | le dépôt GitHub a été créé avec un README : `git pull origin main --allow-unrelated-histories` |
| `Permission denied (publickey)` | la clé SSH n'est pas sur GitHub : `cat ~/.ssh/id_ed25519.pub` et l'ajouter dans **Settings**, **SSH and GPG keys** |
| GitHub me demande un mot de passe | le dépôt a été cloné en HTTPS : `git remote set-url origin git@github.com:pseudo/depot.git` |
| Vim s'est ouvert et je suis coincé | `Échap`, puis `:wq` et `Entrée` |
| Je suis en *detached HEAD* | `git switch main`, ou `git switch -c <nouvelle-branche>` pour garder ce que vous y avez fait |
| J'ai supprimé une branche avec `-D` | `git reflog`, retrouver l'identifiant, puis `git branch <nom> <id>` |
| Je dois changer de branche mais j'ai du travail en cours | `git stash`, changer de branche, revenir, `git stash pop` |
| Je suis perdu au milieu d'une fusion | `git merge --abort` |
| `git status` montre des centaines de fichiers | `git init` lancé dans le mauvais dossier (le home ?) : vérifiez avec `git rev-parse --show-toplevel` |